Curso: Engenharia de Software	**Série:** 5	**Turma:**	**Turno:** N
Professor(a): Douglas Lopes Farias	**Horário:** 19:00		
Acadêmico (a):		**RA:**	
Disciplina: Estrutura de Dados, Pesquisa e Ordenação		**Data:**	

Prova	Prova Prática	Atividades de estudo programadas (AEP)	Prova integrada	Nota final do bimestre
5.0	**3.0**	**1.0**	**1.0**	**10.0**

INSTRUÇÕES PARA REALIZAÇÃO DA PROVA:

⇒ Os dados do cabeçalho deverão ser preenchidos com letra maiúscula. E as questões deverão ser respondidas com letra legível.

⇒ É vedado, durante a prova, o porte e/ou o uso de aparelhos sonoros, fonográficos, de comunicação ou de registro eletrônico ou não, tais como: notebooks, celulares, tabletes e similares.

⇒ A prova é individual e sem consulta, deverá ser respondida a caneta azul ou preta. Prova escrita a lápis não dá direito à revisão. Não é permitido o uso de corretivo.

⇒ É obrigatória a permanência do acadêmico 1 (uma) hora em sala de aula após o início da prova.

⇒ Não será permitida a entrada na sala de aula após 10 minutos do início da prova.

⇒ É obrigatória a assinatura da lista de presença impressa na qual constam RA, nome e curso.

⇒ O valor de cada questão está ao lado da mesma.

⇒ Todas as respostas devem constar no espaço destinado e autorizado pelo professor, à resposta.

⇒ Em caso de qualquer irregularidade comunicar ao Professor ou fiscal de sala.

⇒ Ao término da prova, levante o braço e aguarde atendimento do professor ou do fiscal.

1ºbim.		2ºbim.		1ªsub.		3ºbim.		4ºbim.		2ªsub.	

Questão	A	B	C	D	E
8					
9					
10					

Questão 01 (0,2 ponto) De acordo com os conceitos fundamentais de **Guimarães e Lages (1988)**, uma árvore binária é definida como uma estrutura hierárquica onde cada elemento, denominado nó, possui um grau de saída que pode variar entre 0, 1 ou 2. No desenvolvimento de algoritmos em linguagem C, essa estrutura é viabilizada pelo uso de ponteiros que referenciam os descendentes à esquerda e à direita. Quando um nó apresenta ambos os ponteiros (esq e dir) com valor nulo (**NULL**), ele é classificado na literatura como um nó folha ou terminal.

ANALISE o trecho de código a seguir:

```
Nodo* create(int valor){
    Nodo *n = malloc(sizeof(Nodo));
    n->valor = valor;
    n->esq = NULL;
    n->dir = NULL;
    return n;
}
main() {
    Nodo *nodo5 = create(5);
    Nodo *nodo2 = create(2);
    Nodo *nodo1 = create(1);
    Nodo *nodo8 = create(8);
    Nodo *nodo4 = create(4);
    nodo5->esq = nodo2;
    nodo2->esq = nodo1;
    nodo5->dir = nodo8;
    nodo2->dir = nodo4;
}
```

Fonte: elaborado pelo professor (2026)

Após a leitura realizada do código fonte, **IDENTIFIQUE** quais nós específicos atuam como folhas na árvore resultante, apresente a(s) **JUSTIFICATIVA de** sua resposta.

Questão 2 (0,8 ponto) Conforme descrito por **Cormen (2012)**, o percurso em profundidade do tipo pós-ordem (*post-order*) é um processo sistemático de visitação onde as **subárvores** de um nó são processadas integralmente antes que a própria raiz seja visitada. Esse método é essencial em operações complexas, como a **desalocação** de memória ou avaliação de expressões aritméticas representadas por árvores.

No código a seguir é apresentado, nós instanciados e vinculados manualmente, onde o nodo5 assume o papel de raiz.

```
Nodo* create(int valor){
    Nodo *n = malloc(sizeof(Nodo));
    n->valor = valor;
    n->esq = NULL;
    n->dir = NULL;
    return n;
}
void imprimir(Nodo *n){
    if(n == NULL)
    return;
    imprimir(n->esq);
    printf("\n%d", n->valor);
    imprimir(n->dir);
}
main() {
    Nodo *nodo5 = create(5);
    Nodo *nodo2 = create(2);
    Nodo *nodo1 = create(1);
    Nodo *nodo8 = create(8);
    Nodo *nodo4 = create(4);
    nodo5->esq = nodo2;
```

```
    nodo2->esq = nodo1;
    nodo5->dir = nodo8;
    nodo2->dir = nodo4;
    imprimir(nodo5);
}
```

Fonte: elaborado pelo professor (2026)

APRESENTE qual seria a sequência **EXATA** de valores impressos em tela após ser executado. **EXPLIQUE** o passo a passo da travessia recursiva até atingir a raiz final.

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Questão 3 (0,5 ponto) Na visão de **Ziviani (2013),** a topologia de uma árvore binária pode ser analisada através da métrica de níveis, onde o nível de um nó é definido pela distância ou quantidade de arestas existentes entre ele e a raiz da estrutura. Além disso, as relações de parentesco definem que um nó que aponta para outro é seu pai, enquanto o referenciado é seu filho.

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2013.

No trecho de código a seguir, **OBSERVE** que o nodo5 referência o nodo2, que por sua vez referência o nodo1.

```
Nodo* create(int valor){
    Nodo *n = malloc(sizeof(Nodo));
    n->valor = valor;
    n->esq = NULL;
    n->dir = NULL;
    return n;
}
main() {
    Nodo *nodo5 = create(5);
    Nodo *nodo2 = create(2);
    Nodo *nodo1 = create(1);
    Nodo *nodo8 = create(8);
    Nodo *nodo4 = create(4);
    nodo5->esq = nodo2;
    nodo2->esq = nodo1;
    nodo5->dir = nodo8;
    nodo2->dir = nodo4;
}
```

Fonte: elaborado pelo professor (2026)

Com base na organização montada, **DETERMINE** em qual nível se encontra o nodo1 (considerando a raiz como nível 1) e **IDENTIFIQUE** seu nó pai direto, **DESCREVENDO** o caminho percorrido desde a raiz.

Questão 4 (0,2 ponto) A obra de **Guimarães e Lages (1988)** enfatiza que a robustez de um algoritmo que manipula estruturas dinâmicas reside na correta gestão de seus ponteiros. Na função **create** apresentada, cada novo nó alocado recebe explicitamente o valor NULL em seus campos **esq** e **dir**, garantindo que o programa possua um marcador de fim de caminho confiável na memória.

DISCORRA sobre a importância técnica dessa inicialização e **EXPLIQUE** o que aconteceria se o programador deixasse os ponteiros com "lixo de memória" ao tentar executar uma função recursiva sobre essa árvore.

```
Nodo* create(int valor){
    Nodo *n = malloc(sizeof(Nodo));
    n->valor = valor;
    n->esq = NULL;
    n->dir = NULL;
    return n;
}
main() {
    Nodo *nodo5 = create(5);
    Nodo *nodo2 = create(2);
    Nodo *nodo1 = create(1);
    Nodo *nodo8 = create(8);
    Nodo *nodo4 = create(4);
    nodo5->esq = nodo2;
    nodo2->esq = nodo1;
    nodo5->dir = nodo8;
    nodo2->dir = nodo4;
}
```

Fonte: elaborado pelo professor (2026)

Questão 5 (0,8 ponto) Considerando a teoria de Ziviani (2013), uma Árvore Binária de Busca (BST) deve garantir que todos os nós na **subárvore** esquerda de um nó possuam chaves menores que, e os da direita, chaves maiores. O código **main** atual executa essa organização de forma estática, o que contraria os princípios de escalabilidade da engenharia de software de **Hirama (2011)**, exigindo uma função de inserção autônoma.

DESCREVA logicamente os passos para implementar a função inserir(Nodo** raiz, int valor), explicando como a comparação de valores e a recursividade devem ser utilizadas para localizar o ponto correto de inserção.

Questão 6 (0,5 ponto) Segundo **Ziviani (2013)**, percorrer uma árvore é processar cada nó exatamente uma vez, sendo a recursão a estratégia mais natural para tal tarefa. Melo (2003) destaca que o sucesso de uma rotina recursiva depende de uma "condição de parada" clara e do uso correto da pilha de execução (*stack*) para gerenciar o retorno do fluxo de controle aos estados anteriores.

ANALISE o algoritmo imprimir(Nodo *n) fornecido e **EXPLIQUE** a execução do código e qual problema ele apresenta:

```
void imprimir(Nodo *n){
    imprimir(n->esq);
    printf("\n%d", n->valor);
    imprimir(n->dir);
}
```

Fonte: elaborado pelo professor (2026)

Questão 7 (0,5 ponto) Segundo **Ziviani (2013)**, percorrer uma árvore é processar cada nó exatamente uma vez, sendo a recursão a estratégia mais natural para tal tarefa. Melo (2003) destaca que o sucesso de uma rotina recursiva depende de uma

"condição de parada" clara e do uso correto da pilha de execução (*stack*) para gerenciar o retorno do fluxo de controle aos estados anteriores.

ANALISE o algoritmo imprimir(Nodo *n) fornecido e **REESCREVA** o código com a alteração para que a impressão possa ocorrer em ordem decrescente:

```
void imprimir(Nodo *n){
    if(n == NULL)
    return;
    imprimir(n->esq);
    printf("\n%d", n->valor);
    imprimir(n->dir);
}
```

Fonte: elaborado pelo professor (2026)

Questão 8 (0,5 ponto) Analise o código base sob a ótica da qualidade de software proposta por **Hirama (2011)** e da eficiência de algoritmos de **Ziviani (2013).** A função **create** isola a alocação de memória, enquanto o **main** estabelece conexões

manuais entre os nós 5, 2, 1, 8 e 4. Considere as afirmações sobre manutenibilidade e topologia.

AVALIE as asserções (I, II, III e IV) apresentadas no corpo da questão e **ASSINALE** a alternativa que indica quais estão tecnicamente corretas quanto ao uso de ponteiros e encapsulamento.

```
Nodo* create(int valor){
    Nodo *n = malloc(sizeof(Nodo));
    n->valor = valor;
    n->esq = NULL;
    n->dir = NULL;
    return n;
}
main() {
    Nodo *nodo5 = create(5);
    Nodo *nodo2 = create(2);
    Nodo *nodo1 = create(1);
    Nodo *nodo8 = create(8);
    Nodo *nodo4 = create(4);
    nodo5->esq = nodo2;
    nodo2->esq = nodo1;
    nodo5->dir = nodo8;
    nodo2->dir = nodo4;
}
```

Fonte: elaborado pelo professor (2026)

Sobre o código acima, **ANALISE** as asserções sobre a qualidade do código e a estrutura de dados fundamentada em **Hirama (2011)** e **Ziviani (2013)**. Em seguida, **ASSINALE** a alternativa correta.

I. A função "*create"* promove criação do nó, melhorando a manutenibilidade do software.

II. A árvore montada no "*main*" é uma Árvore Binária de Busca (BST) válida, pois para todo nó, o filho à esquerda é menor e o à direita é maior que o pai.

III. A ordem de inserção de forma manual dos nós no método "*main"* podem interferir

na topologia final da árvore, porém no código acima as inserções foram realizadas de forma ordenada.
IV. O uso de ponteiros permite que a árvore seja uma estrutura estática

É correto o que se afirma em:
A) I e IV, apenas.
B) I, II e IV, apenas.
C) II e III, apenas.
D) IV, apenas.
E) I, II e III, apenas.

Questão 9 (0,5 ponto) Baseando-se na bibliografia de **Guimarães e Lages (1988)** e nos conceitos de linguagem C de **Kernighan e Ritchie (1989),** a definição de um tipo abstrato de dados para árvores exige a compreensão de como a memória é reservada. A estrutura Nodo utiliza ponteiros autorreferenciais para construir a malha hierárquica através de alocação dinâmica.

ANALISE as afirmações sobre o uso da função malloc e a estrutura dos campos esq e dir, identificando se a alocação ocorre na área de *heap* ou *stack* do sistema operacional. Em seguida **ASSINALE** a alternativa correta.

Sobre a estrutura struct nodo definida no código, analise as afirmações:
I. O campo valor armazena o dado, enquanto esq e dir são ponteiros para os filhos.
II. A estrutura permite a criação de uma árvore binária onde cada nó tem no máximo dois filhos.
III. O uso de malloc no método create aloca o nodo na memória e automaticamente organiza a ordem da arvore.

Está correto o que se afirma em:
a) I, apenas.
b) II, apenas.
c) I e II, apenas.
d) II e III, apenas.
e) I, II e III.

Questão 10 (0,5 ponto) Segundo **Melo (2003) e Ziviani (2013)**, a recursividade é um mecanismo onde uma função chama a si mesma para resolver subproblemas de mesma natureza. Para que o fluxo de controle seja encerrado corretamente, é imprescindível a existência de uma "condição base" ou "caso trivial". Internamente, cada chamada recursiva suspende a execução da função atual e armazena seu estado (variáveis e endereço de retorno) em uma estrutura de dados do tipo LIFO (*Last-In, First-Out*), conhecida como pilha de execução ou *stack*.

Conforme **Cormen (2012) e Guimarães e Lages (1988),** uma Árvore Binária de Busca (BST) é uma estrutura cujos nós são organizados de modo que, para qualquer nó, todos os valores na sua subárvore esquerda sejam estritamente menores que o valor de, e todos os valores na sua subárvore direita sejam maiores. Essa propriedade de ordenação deve ser mantida em todos os níveis da árvore, permitindo que operações de busca e inserção sejam realizadas de forma eficiente.

Com base nos textos acima e nos conceitos de estruturas de dados, **ANALISE** as assertivas a seguir:

I.O sucesso de um algoritmo recursivo em uma BST depende da condição de parada; caso o ponteiro nulo (NULL) não seja verificado, o programa pode sofrer um estouro de pilha e entrar em um looping infinito.

II. Em uma BST perfeitamente organizada, o percurso do tipo "em-ordem" (*in-order*), implementado de forma recursiva, visitará os nós em uma sequência numérica apenas do lado direito da arvore, do lado esquerdo visitara os nodos em uma sequência de ponteiros.

III. A principal vantagem da recursividade na manipulação de árvores é a economia de memória RAM, uma vez que a pilha de execução analisa o ponteiro (n->inferior) para parar a execução em algum momento.

IV. A propriedade fundamental de uma BST dita que o nó raiz deve ser sempre o maior valor da estrutura, para que os descendentes possam ser distribuídos nas subárvores e automaticamente tornando a arvore balanceada.

V. Uma árvore binária é uma estrutura de dados baseada em nós(nodos) organizados hierarquicamente, otimizado a busca, inserção e remoção.

ASSINALE a alternativa que responde corretamente quais assertivas são verdadeiras:

A) Apenas I e II são verdadeiras.

B) Apenas I e V são verdadeiras.

C) Apenas II, III e IV são verdadeiras.

D) Apenas I, IV e V são verdadeiras.

E) Apenas I é verdadeira.